Open in Colab

After clicking the "Open in Colab" link, copy the notebook to your own Google Drive before getting started, or it will not save your work

BYU CS 180 Lab 3: Intro to Pandas

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Introduction:

Welcome to your first pandas lab!

Much of this lab has been adapted from this link pandas introduction lab from the ACME major. Feel free to read through it and use it as you complete this lab.

Lab Objective:

The goal of this lab is for you to become more comfortable in Python's pandas library. We'll introduce you to pandas data structures, syntax, and powerful capacity.

Important Hints

Notice that most of the functions we learn about in this lab return new values. In order to save these values, we must store them.

For example, `df.drop(columns=['column1'])` will drop the column1 from df, but we must call `df = df.drop(columns=['column1'])` to store the changed dataframe. Alternatively, you can set the `inplace` argument to `True` to save changes to the df without explicite reassignment.

Series:

The following cell creates a pandas series, which is essentially a list with an index for each entry in the list. The index is generally used as a label for the data.

```
# Run the below cell to create a new Series:
math = pd.Series([80,96,78,59],['Mark','Barbara','Eleanor','David'])
print(math)
```

```
Mark       80
Barbara    96
Eleanor    78
David      59
dtype: int64
```

Notice that each element in the above series 'math' is a number 1-100, and each of these elements are labeled with a name. The dtype (data type) of this series is an int64. Let's say these numbers represent each student's grade in their math class.

Exercise 1: Pandas Series

Create a pandas series of type int64 called 'english' storing our four student's english grades:

Mark -> 90

Barbara -> 87

Eleanor -> 97

David -> 65

```
#Create a pandas series below:
english = pd.Series([90,87,97,65],['Mark','Barbara','Eleanor','David'])
```

```
english
```

	0
Mark	90
Barbara	87
Eleanor	97
David	65

dtype: int64

DataFrame

The next, and most important, data structure in pandas is the DataFrame. A DataFrame is a collection of multiple Series objects, and it is essentially a 2d array/list with each row labeled by an index, and each column labeled as a column.

Below we initialize a DataFrame, 'simple_grades', using the 'math' and 'english' Series that we created above.

```
simple_grades = pd.DataFrame({"Math": math, "English": english})
print(simple_grades)
```

```
         Math  English
Mark       80       90
Barbara    96       87
Eleanor    78       97
David      59       65
```

Notice that we now have numbers that are labelled twice. Mark's English grade is an 90. Eleanor's Math grade is a 78.

We can also initialize a DataFame using a NumPy array, since pandas is built on top of NumPy. We do that below and call it 'grades'.

```
data = np.array([[52.0, 73.0], [10.0, 39.0], [35.0, np.nan], [np.nan, 26.0],[np.nan,99.0],[81.0,68.0]])
grades = pd.DataFrame(data, columns = ['Math', 'English'], index = ['Barbara','David','Eleanor','Greg','Lauren','Mark'])
grades
```

	Math	English
Barbara	52.0	73.0
David	10.0	39.0
Eleanor	35.0	NaN
Greg	NaN	26.0
Lauren	NaN	99.0
Mark	81.0	68.0

Next steps: Generate code with grades | View recommended plots | New interactive sheet

```
# look at the column labels of grades
print(grades.columns)

# look at the index labels of grades
print(grades.index)

# look at the values (2d array) of grades
print(grades.values)
```

```
Index(['Math', 'English'], dtype='object')
Index(['Barbara', 'David', 'Eleanor', 'Greg', 'Lauren', 'Mark'], dtype='object')
[[52. 73.]
 [10. 39.]
 [35. nan]
 [nan 26.]
 [nan 99.]
 [81. 68.]]
```

Exercise 2:

To access data in a DataFrame, we use the .loc and the .iloc indexers.

The .loc index selects rows and columns based on their labels. In the below examples, we are looking at the rows of 'David' and 'Greg', while only viewing the 'Math' column. Notice that a list of indices is used to view multiple rows by name.

```
grades.loc[['David','Greg'],'Math']
```

	Math
David	10.0
Greg	NaN

dtype: float64

The .iloc method selects rows and columns based on their integer position

```
grades.iloc[[1,3],0]
```

	Math
David	10.0
Greg	NaN

dtype: float64

Use `.loc` to print Eleanor and Mark's grades in both English and Math

```
#Enter the code here to print both Eleanor and Marks Grades
grades.loc[['Eleanor','Mark']]
```

	Math	English
Eleanor	35.0	NaN
Mark	81.0	68.0

You can access an entire column of a DataFrame by using simple square brackets and the name of the column.

```
grades['Math']
```

	Math
Barbara	52.0
David	10.0
Eleanor	35.0
Greg	NaN
Lauren	NaN
Mark	81.0

dtype: float64

Using the same logic, we can also create a new column using either a numpy array, a list, or a single value.

```
grades['History'] = np.random.randint(0, 100, 6)
grades['History'] = 100
```

To view the beginning of a DataFrame, we can use .head(n). This makes it a lot easier to get an idea of what the data look like without printing the entire dataframe (especially when the df is huge!).

```
grades.head(3)
```

	Math	English	History
Barbara	52.0	73.0	100
David	10.0	39.0	100
Eleanor	35.0	NaN	100

Next steps: Generate code with grades | View recommended plots | New interactive sheet

You can use .reindex to change the order of either the rows or columns, and .sort_values to sort the DataFrame by a specified column value.

```
grades.reindex(columns = ['English', 'Math', 'History'])
grades.sort_values('Math', ascending = False)
```

	Math	English	History
Mark	81.0	68.0	100
Barbara	52.0	73.0	100
Eleanor	35.0	NaN	100
David	10.0	39.0	100
Greg	NaN	26.0	100
Lauren	NaN	99.0	100

You can also drop columns from a dataframe by using df.drop(columns=[])

```
grades.drop(columns = ['Math'])
```

	English	History
Barbara	73.0	100
David	39.0	100
Eleanor	NaN	100
Greg	26.0	100
Lauren	99.0	100
Mark	68.0	100

Exercise 3: Girlfriend Vs. Fortnite

The costs.csv downloaded earlier contains an estimate of my costs over the past few semesters.

- Read in the costs.csv file
- Add a column called 'girlfriend' with all values set to 500
- Reindex the columns such that the amount spent on rent is the first column and the other columns stay in the same order
- Sort the DataFrame in descending order based on how much I spent on fortnite_skins
- Reset all the values in the rent column to 1000

```
#Girl Friend Data
!curl -o costs.csv https://raw.githubusercontent.com/wingated/cs180_labs/main/costs.csv
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   125  100   125    0     0    291      0 --:--:-- --:--:-- --:--:--   292
```

```
#Enter the code for the question above here:
costs = pd.read_csv('costs.csv')
costs['girlfriend'] = 500
new_order = ['rent'] + [col for col in costs.columns if col != 'rent']
costs_reordered = costs.reindex(columns=new_order)
costs_reordered = costs_reordered.sort_values('fortnite_skins', ascending = False)
costs_reordered['rent'] = 1000
costs_reordered
```

	rent	books	food	fortnite_skins	girlfriend
2	1000	300	775	40	500
3	1000	312	750	18	500
4	1000	330	712	16	500
0	1000	385	800	15	500
1	1000	280	700	10	500
5	1000	120	900	5	500

Next steps: Generate code with costs_reordered | View recommended plots | New interactive sheet

Exercise 4: Means on Columns

Calculate the mean cost of each column in the costs DataFrame in the cell below. (Hint: use the DataFrame.mean() function!)

```
#Mean cost of each column
for col in costs_reordered.columns:
  print(f"The means for column {col} is {costs_reordered[col].mean()}")
```

```
The means for column rent is 1000.0
The means for column books is 287.8333333333333
The means for column food is 772.8333333333334
The means for column fortnite_skins is 17.333333333333332
The means for column girlfriend is 500.0
```

Double-click (or enter) to edit

Exercise 5: Supplements

Now we will return to the grades DataFrame that we created earlier.

Dealing with missing data is a difficult topic in data science. The pandas default for missing values is NaN. These can be difficult to deal with because any operation (addition, multiplication, etc) involving an NaN value will always result in an NaN, so finding the mean of a column or adding up all the rows will be meaningless.

What do we do with NaN values? The answer is always: it depends, but we should also ask: why do we have missing values? It could be that some people only filled out half the survey, it could be that the data should read 0.0 but it wasn't filled out. It could mean (in our example) that the student isn't enrolled in that class. It could be many reasons, and we should always figure them out first!

In pandas we can do a couple things with NaN values.

To drop all rows containing NaN values, we can simply call DataFrame.dropna()

Or we could fill the NaN values with a specified value, like 0.0:

```
grades.fillna(0.0)
```

	Math	English	History
Barbara	52.0	73.0	100
David	10.0	39.0	100
Eleanor	35.0	0.0	100
Greg	0.0	26.0	100
Lauren	0.0	99.0	100
Mark	81.0	68.0	100

The supplements.csv downloaded below contains vitamin information (in mg) for 20 different supplements I'm considering as I get ready for summer:

- Read in the supplements.csv file
- Fill all the na values using method='bfill' (HINT: put method='bfill' in the function call! Google it if you're confused)
- Sort the DataFrame by my most important vitamin, vitamin b6, in descending order
- Use .drop() to create a new df, subset_df, containing all the vitamins in the supplements file except vitamin_d
- Create a boxplot of all columns in subset_df (hint - make sure to call plt.show() at the end!!)

```
!curl -o supplements.csv https://raw.githubusercontent.com/porterjenkins/CS180/main/data/supplements.csv
```

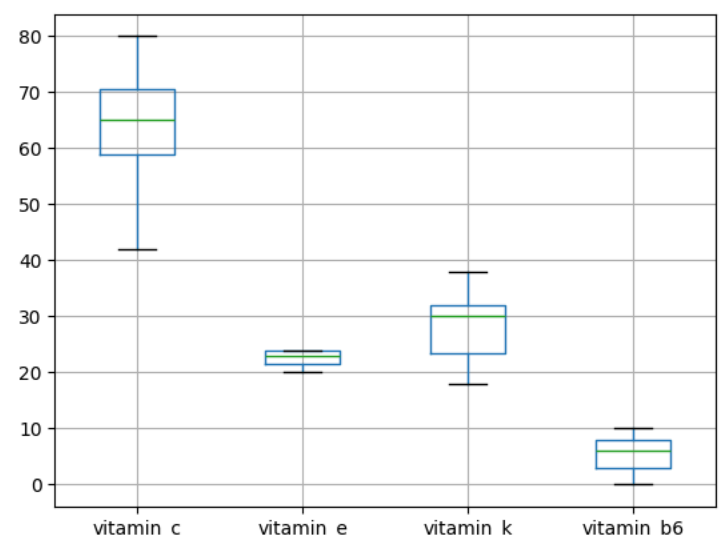
```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   338  100   338    0     0    669      0 --:--:-- --:--:-- --:--:--   670
```

```
#Enter your code in here for question #5
supplement = pd.read_csv('supplements.csv')
supplement.fillna(method='bfill')
supplement.sort_values('vitamin_b6', ascending = False)
```

```
<ipython-input-20-396916604e61>:3: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future ve
  supplement.fillna(method='bfill')
```

	vitamin_c	vitamin_d	vitamin_e	vitamin_k	vitamin_b6
15	77.0	790.0	23.0	23.0	10.0
14	80.0	NaN	NaN	38.0	10.0
3	49.0	506.0	20.0	37.0	9.0
9	59.0	476.0	20.0	32.0	9.0
10	65.0	585.0	24.0	18.0	8.0
5	69.0	417.0	24.0	38.0	8.0
7	59.0	768.0	24.0	31.0	8.0
11	80.0	516.0	23.0	30.0	7.0
18	NaN	517.0	24.0	20.0	6.0
0	NaN	547.0	21.0	25.0	6.0
1	65.0	744.0	21.0	25.0	6.0
6	51.0	556.0	20.0	NaN	5.0
17	42.0	764.0	22.0	NaN	5.0
12	59.0	591.0	23.0	30.0	3.0
19	68.0	770.0	24.0	31.0	3.0
8	69.0	439.0	24.0	37.0	1.0
16	71.0	771.0	24.0	18.0	1.0
13	60.0	549.0	22.0	32.0	0.0
4	43.0	520.0	24.0	23.0	0.0
2	71.0	664.0	22.0	30.0	NaN

```
subset_df = supplement.drop(columns = ['vitamin_d'])
subset_df.boxplot()
plt.show()
```



Exercise 6

Write something that you noticed in the supplements data. Feel free to poke around, plot some more things, and find something interesting!

```
# Exploratory Data Analysis

supplement.describe()
```

	vitamin_c	vitamin_d	vitamin_e	vitamin_k	vitamin_b6
count	18.000000	19.000000	19.000000	18.000000	19.000000
mean	63.166667	604.736842	22.578947	28.777778	5.526316
std	11.490405	125.915687	1.538968	6.620428	3.339468
min	42.000000	417.000000	20.000000	18.000000	0.000000
25%	59.000000	516.500000	21.500000	23.500000	3.000000
50%	65.000000	556.000000	23.000000	30.000000	6.000000
75%	70.500000	754.000000	24.000000	32.000000	8.000000
max	80.000000	790.000000	24.000000	38.000000	10.000000

```
mean = supplement.mean()
median = supplement.median()
print(f'The mean of each columns is\n{mean} \n')

print(f'The median of each columns is\n{median}')
```

```
The mean of each columns is
vitamin_c      63.166667
vitamin_d     604.736842
vitamin_e      22.578947
vitamin_k      28.777778
vitamin_b6      5.526316
dtype: float64

The median of each columns is
vitamin_c      65.0
vitamin_d     556.0
vitamin_e      23.0
vitamin_k      30.0
vitamin_b6      6.0
dtype: float64
```

Enter something cool that you found out here.